

1 Алгоритм k-ближайших соседей (k-NN)

1.0.1 Математическая формулировка

Пусть задана обучающая выборка $X^\ell = \{(x_1, y_1), \dots, (x_\ell, y_\ell)\}$, где $x_i \in \mathbb{R}^d$ — векторы признаков, а y_i — целевые значения. Для произвольного объекта x определим порядок близости объектов обучающей выборки:

$$\rho(x, x_{(1)}) \leq \rho(x, x_{(2)}) \leq \dots \leq \rho(x, x_{(\ell)})$$

где $\rho(x, z)$ — функция расстояния (метрика), а $x_{(i)}$ — i -й ближайший сосед объекта x .

Важное условие: Использование метрик требует обязательной нормировки признаков, так как признаки с большими абсолютными значениями будут доминировать в расчете расстояния.

1.0.2 Классификация и Регрессия

- **Классификация:** Ответ — наиболее часто встречающийся класс среди k соседей (мода). При взвешенном голосовании: $a(x) = \operatorname{argmax}_{y \in Y} \sum_{i=1}^k w_i [y_{(i)} = y]$.
- **Регрессия:** Ответ — среднее арифметическое ответов соседей: $a(x) = \frac{1}{k} \sum_{i=1}^k y_{(i)}$. Во взвешенном варианте: $a(x) = \frac{\sum w_i y_{(i)}}{\sum w_i}$.

1.0.3 Выбор параметров

- **Число соседей k :**
 - При малых k модель имеет **низкое смещение (bias)**, но **высокий разброс (variance)**. Риск переобучения.
 - При больших k модель имеет **высокое смещение**, но **низкий разброс**. Риск недообучения.
- **Методы выбора:** Обычно k подбирают на кросс-валидации или методом Leave-One-Out. Также используют «метод локтя» (анализ графика ошибки).

Выбор k — это классический поиск компромисса между недообучением и переобучением. Обычно k подбирают на кросс-валидации.

2 Метрики и функции расстояния

- **Семейство Минковского (L_p):** Общая формула: $\rho(x, z) = \left(\sum_{i=1}^d |x_i - z_i|^p \right)^{1/p}$.
 - L_1 (**Manhattan**): $\sum_{i=1}^d |x_i - z_i|$. Менее чувствительна к выбросам, часто лучше работает в пространствах высокой размерности.
 - L_2 (**Euclidean**): $\sqrt{\sum_{i=1}^d (x_i - z_i)^2}$. Стандартное расстояние «по прямой». Чувствительно к большим отклонениям в отдельных признаках.
 - L_∞ (**Chebyshev**): $\max_i |x_i - z_i|$. Расстояние определяется максимальным различием по одной из координат.

- **Косинусное расстояние:** $d_{\cos}(x, z) = 1 - \frac{\langle x, z \rangle}{\|x\| \|z\|} = 1 - \frac{\sum x_i z_i}{\sqrt{\sum x_i^2} \sqrt{\sum z_i^2}}$. Измеряет угол между векторами, игнорируя их норму (длину). Широко применяется в анализе текстов и NLP.
- **Расстояние Жаккара:** $d_J(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|}$. Используется для сравнения множеств или бинарных признаков (например, наличие слов в документе). Отражает долю общих элементов от их общего количества.

*Помните, что перед использованием L_p метрик данные **обязательно** нужно масштабировать, иначе признак с большим диапазоном значений «поглотит» все остальные.*

3 Взвешенный k-NN

3.1 Веса по рангу (порядковому номеру)

Вес соседа зависит не от расстояния до него, а от его места в списке ближайших соседей.

- **Линейно убывающие веса:**

$$w_i = \frac{k + 1 - i}{k}$$

- **Экспоненциально убывающие веса:**

$$w_i = q^i, \quad 0 < q < 1$$

3.2 Веса от расстояния

Вес вычисляется непосредственно как функция от расстояния $\rho(x, x_{(i)})$.

- **Обратное расстояние:**

$$w_i = \frac{1}{\rho(x, x_{(i)}) + \epsilon}$$

- **Относительное расстояние:**

$$w_i = \frac{\rho(x, x_{(k+1)}) - \rho(x, x_{(i)})}{\rho(x, x_{(k+1)}) - \rho(x, x_{(1)})}$$

3.3 Ядерное сглаживание

Вес соседа определяется через функцию ядра $K(z)$:

$$w_i = K\left(\frac{\rho(x, x_i)}{h}\right)$$

- **Типы ядер:**

- **Епанечникова:**

$$K(z) = \frac{3}{4}(1 - z^2)[|z| \leq 1]$$

- **Квартическое:**

$$K(z) = \frac{15}{16}(1 - z^2)^2[|z| \leq 1]$$

- **Гауссовское:**

$$K(z) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}z^2}$$

3.4 Ядерная регрессия (Формула Надарая-Ватсона)

Это обобщение k-NN для задачи регрессии, где предсказание — это взвешенное среднее ответов всех объектов обучающей выборки:

$$a(x) = \frac{\sum_{i=1}^n y_i K\left(\frac{\rho(x, x_i)}{h}\right)}{\sum_{i=1}^n K\left(\frac{\rho(x, x_i)}{h}\right)}$$

3.5 Оптимизация поиска ближайшего соседа

Для работы с большими данными ($N \gg 1$) используются методы, позволяющие избежать полного перебора объектов.

- **Точные методы:**
 - **k-d деревья:** Разбивают пространство по признакам. Эффективны только при малых d (< 20). При высокой размерности деградируют до $O(N)$.
 - **Ball trees:** Разбивают пространство на гиперболы. Работают стабильнее k-d деревьев в пространствах средней размерности.
- **Приближенные методы (ANN):**
 - **LSH (Locality-Sensitive Hashing):** Хеширование, при котором близкие объекты с высокой вероятностью попадают в один бакет. Поиск идет только внутри бакета.
 - **HNSW (Hierarchical Navigable Small World):** Построение графа, где вершины — объекты, а ребра соединяют соседей. Иерархическая структура позволяет быстро выходить в нужную область пространства и уточнять результат.
 - **Inverted File Index (IVF):** Пространство делится на кластеры (ячейки Вороного), и поиск ведется только в ближайших к запросу кластерах.

Antigravity: В реальном продакшене (Faiss, HNSW) почти всегда используются именно ANN, так как точный поиск на миллионах объектов невозможен.